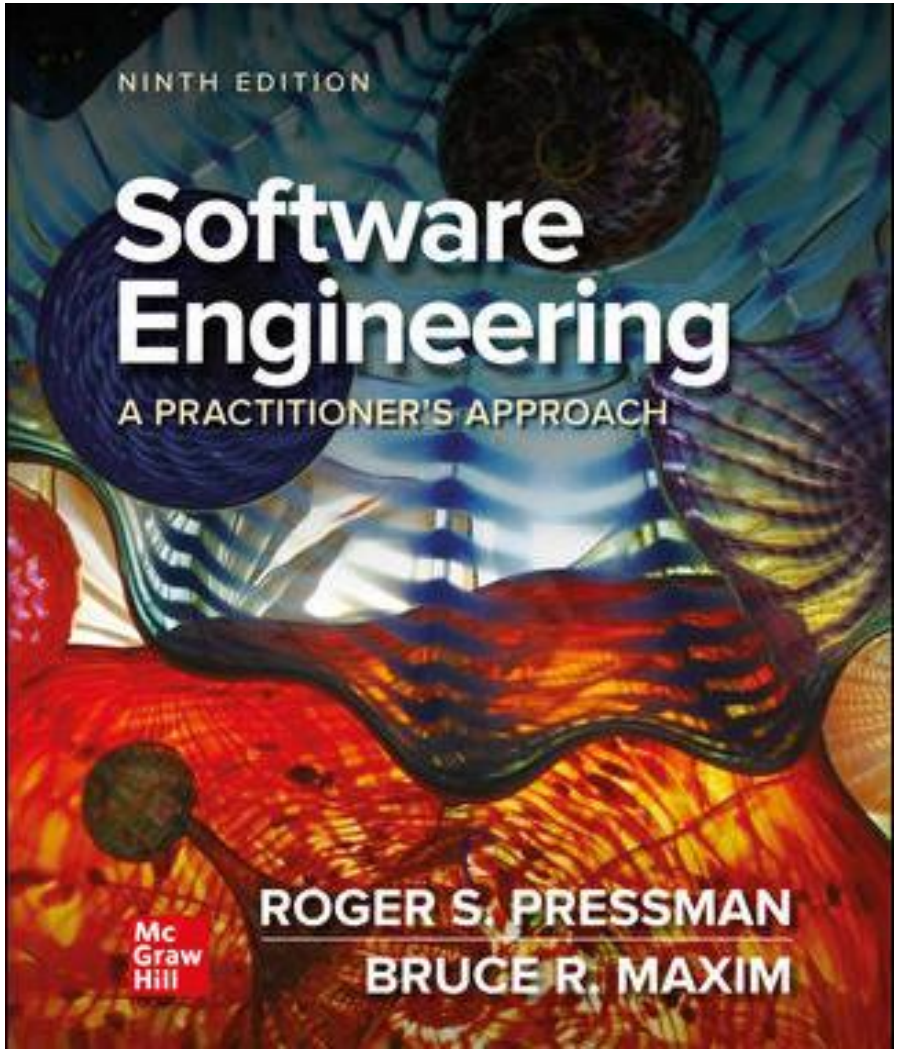
Mc Graw Hill Education

Because learning changes everything.®

Chapter 21

Software Testing – Specilzed Testing for Mobility

Part 3 – Quality and Security



Creating a Mobile Test Plan

- Do you have to build a fully functional prototype before you test with users?
- Should you test with the user's device or provide a device for testing?
- What devices and user groups should you include in testing?
- When is lab testing versus remote testing appropriate.

Mobile Testing Guidelines 1 [1]

- Understand the network landscape and device landscape.
- Conduct testing in uncontrolled real-world test conditions.
- Select the right automation test tool.
- Identify the most critical hardware/ platform combinations to test.

Mobile Testing Guidelines 1 2

- Check the end-to-end functional flow in all possible platforms at least once.
- Conduct performance, GUI, and compatibility testing using actual devices.
- Measure Mobile performance under realistic network load conditions.

Mobile Testing Strategies 1

- **User-experience testing.** Users are involved early in the development process to ensure that the MobileApp lives up to the usability and accessibility expectations of the stakeholders on all supported devices.
- **Device compatibility testing.** Testers verify that the MobileApp works correctly on all required hardware and software combinations.
- **Performance testing.** Testers check nonfunctional requirements unique to mobile devices (for example, download times, processor speed, storage capacity, power availability).

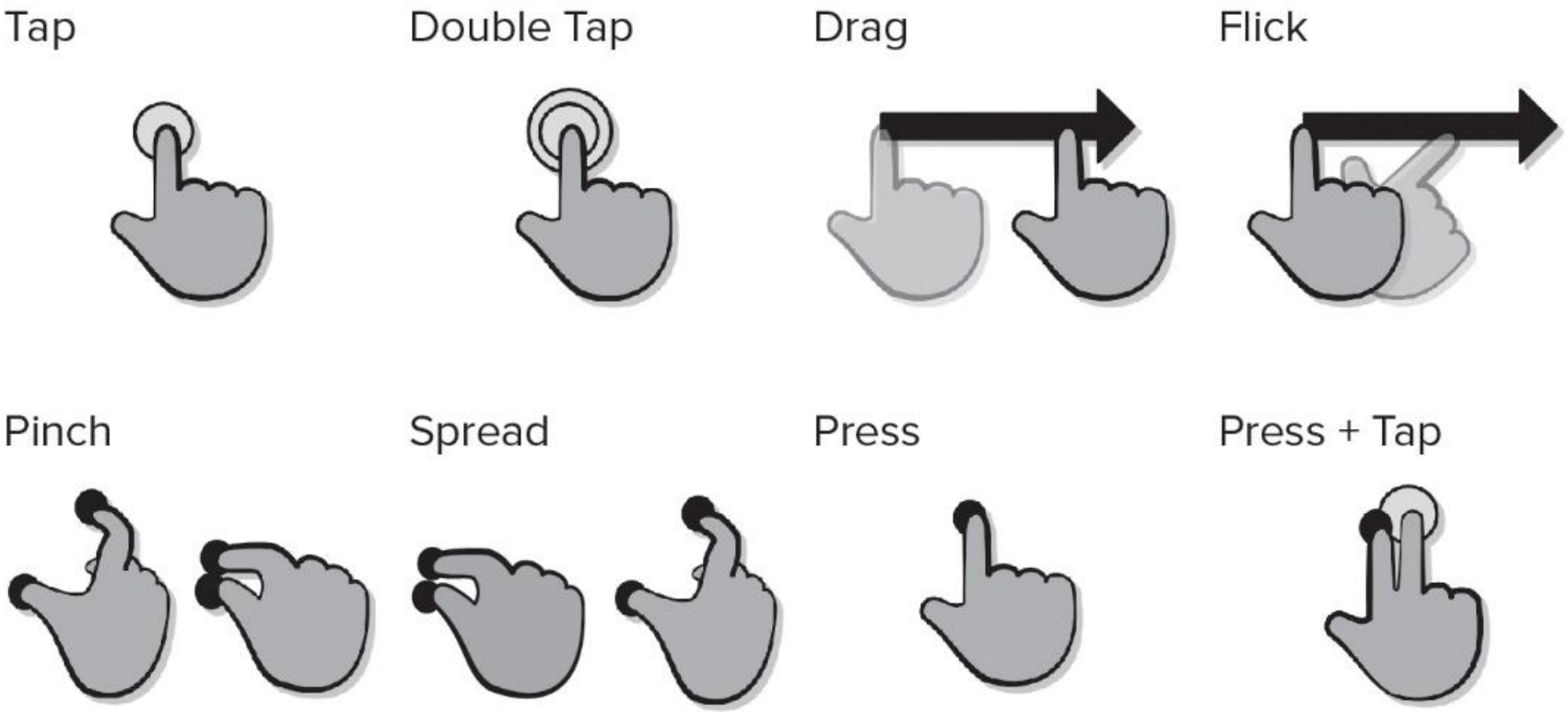
Mobile Testing Strategies 2

- **Connectivity testing.** Testers ensure that the MobileApp can access any needed networks or Web services and can tolerate weak or interrupted network access.
- **Security testing.** Testers ensure that the MobileApp does not compromise the privacy or security requirements of its users.
- **Testing in the wild.** The app is tested under realistic conditions on actual user devices in a variety of networking environments around the globe.
- **Certification testing.** Testers ensure that the MobileApp meets the standards established by the app stores that will distribute it.

UX – Gesture Testing Issues

- Touch screens are ubiquitous on mobile devices and developers have added multitouch gestures as a means of augmenting the user interaction possibilities without losing screen real estate.
- Paper prototypes cannot be used to adequately review the adequacy or efficacy of gestures.
- It's difficult to use automated tools to test gesture interface actions.
- Location of screen objects is affected by screen size and resolution making accurate gesture testing difficult.
- Gestures are hard to log accurately for replay.
- Accessibility testing for visually impaired users is challenging because gesture interfaces typically do not provide either tactile or auditory feedback.

Add Body Slides in this Portion Presentation 1



Access the text alternative for slide images.

UX – Virtual Keyboards

- Virtual keyboard may obscure part of the display screen when activated, it is important to ensure that important screen information is not hidden from the user while typing.
- Virtual keyboards are smaller than personal computer keyboards, it is hard to type with 10 fingers and they provide no tactile feedback.
- The MobileApp must be tested to ensure that it allows easy error correction and can manage mistyped words without crashing.
- ***Predictive technologies*** (that is, autocompletion of partially typed words) are often used with to help expedite user input.
- It is important to test the correctness of the word completions for the natural language chosen by the user.

UX – Voice Input

- Voice input has become an increasingly common method for providing input and commands in hands-busy, eyes-busy situations.
- Using voice commands to control a device impresses a greater cognitive load on the user, than pointing to a screen object or pressing a key.
- Testing the quality and reliability of voice input and recognition needs to take environmental conditions and individual voice variation into account.
- The MobileApp should be tested to ensure that bad input does not crash the MobileApp or the device.
- It is important to log errors to help developers improve the ability of the MobileApp to process speech input.

UX – Alerts and Extraordinary Conditions

- Part of MobileApp testing should focus on the usability issues relating to alerts and pop-up messages.
- Testing should examine the clarity and context of alerts, the appropriateness of their location on the device display screen,
- When foreign languages are involved, verification that the translation from one language to another is correct.
- You should not rely solely on testing in a development environment and you must test MobileApp in the wild on actual devices.
- Apps must recover from faults and resume processing with little or no downtime and in some cases the system must be fault tolerant.
- ***Recovery testing*** is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.

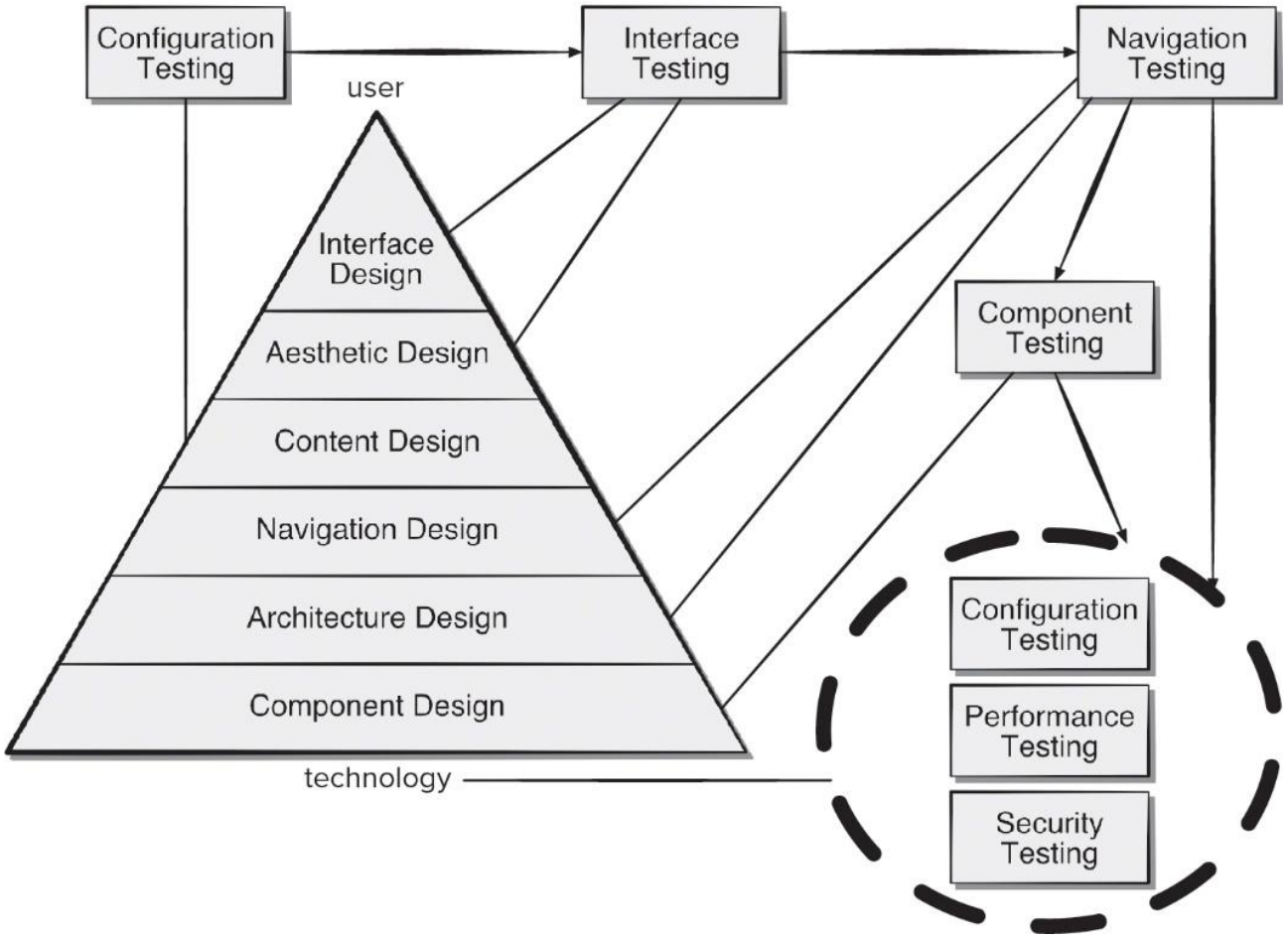
Web App Testing Steps 1 [1]

1. The content model for the WebApp is reviewed to uncover errors.
2. The interface model is reviewed to ensure that all use cases can be accommodated.
3. The design model for the WebApp is reviewed to uncover navigation errors.
4. The user interface is tested to uncover errors in presentation and/or navigation mechanics.
5. Each functional component is unit tested.

Web App Testing Steps 1 2

6. Navigation throughout the architecture is tested.
7. The WebApp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted in an attempt to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users. The results of their interaction with the system are evaluated for errors.

Add Body Slides in this Portion Presentation 2



Access the text alternative for slide images.

Web App – Content Testing

Content testing has three important objectives:

1. to uncover syntactic errors (for example, typos, grammar mistakes) in text-based documents, graphical representations, and other media.
2. to uncover semantic errors (that is, errors in the accuracy or completeness of information) in any content object presented as navigation occurs, and.
3. to find errors in the organization or structure of content that is presented to the end-user.

Assessing Content Sematics

- Is the information factually accurate?
- Is the information concise and to the point?
- Is the layout of the content object easy for the user to understand?
- Can information embedded within a content object be found easily?
- Have proper references been provided for all information derived from other sources?
- Is the information presented consistent internally and consistent with information presented in other content objects?
- Is content offensive, misleading, or an open door to litigation?
- Does the content infringe on existing copyrights or trademarks?
- Does the content contain internal links that supplement existing content? Are the links correct?
- Does the aesthetic style of the content conflict with the aesthetic style of the interface?

Web App – Interface Testing

- Interface features tested to ensure design rules, aesthetics, and content is available to user without error.
- Individual interface mechanisms are tested in a manner that is analogous to unit testing.
- Each interface mechanism is tested within the context of a use-case or NSU for a specific user category.
- Complete interface is tested against selected use-cases and NSUs to uncover errors in interface semantics.
- The interface is tested within a variety of environments (for example, browsers) to ensure that it will be compatible.

Web App – Navigation Testing 1

Answer these questions as each NSU or use case is tested:

- Is the NSU achieved in its entirety without error?
- Is every navigation node (defined for an NSU) reachable within the context of the navigation paths defined for the NSU?
- If the NSU can be achieved using more than one navigation path, has every relevant path been tested?
- If guidance is provided by the user interface to assist in navigation, are directions correct and understandable as navigation proceeds?
- Is there a mechanism (other than the browser "back" arrow) for returning to the preceding navigation node and to the beginning of the navigation path?
- Do mechanisms for navigation within a large navigation node (that is, a long Web page) work properly?

Web App – Navigation Testing 2

- If a function is to be executed at a node and the user chooses not to provide input, can the remainder of the NSU be completed?
- If a function is executed at a node and an error in function processing occurs, can the NSU be completed?
- Is there a way to discontinue the navigation before all nodes have been reached, but then return to where the navigation was discontinued and proceed from there?
- Is every node reachable from the site map? Are node names meaningful to end users?
- If a node within an NSU is reached from some external source, is it possible to process to the next node on the navigation path? Is it possible to return to the previous node on the navigation path?
- Does the user understand his location within the content architecture as the NSU is executed?

Internationalization and Localization

- ***Internationalization*** is the process of creating a software product so that it can be used in several countries and with various languages without requiring any engineering changes.
- ***Localization*** is the process of adapting a software application for use in targeted global regions by adding locale-specific requirements and translating text elements to appropriate languages.
- ***Crowdsourcing*** is a distributed problem-solving model where community members work on solutions to problems posted to the group.
- Crowdsourcing can be used to engage localization testers dispersed around the globe outside of the development environment.

Security Testing

- Designed to probe vulnerabilities of the client-side environment, the network communications that occur as data are passed from client to server and back again, and the server-side environment.
- On the client-side, vulnerabilities can often be traced to pre-existing bugs in browsers, e-mail programs, or communication software.
- On the server-side, vulnerabilities include denial-of-service attacks and malicious scripts that can be passed along to the client-side or used to disable server operations.

Performance Testing

Does the server response time degrade to a point where it is noticeable and unacceptable?

At what point does performance become unacceptable?

What system components are responsible for performance degradation?

What is the average response time for users under a variety of loading conditions?

Does performance degradation have an impact on system security?

Is WebApp reliability or accuracy affected as the load on the system grows?

What happens when loads that are greater than maximum server capacity are applied?

Load Testing

- The intent is to determine how the WebApp and its server-side environment will respond to various loading conditions.

  N, number of concurrent users

  T, number of on-line transactions per unit of time

  D, data load processed by server per transaction

- Overall throughput, *P,* is computed in the following manner:

$$P = N \times T \times D$$

Stress Testing

Stress testing for mobile apps attempts to find errors that occur under extreme operating conditions such as:

1. running several mobile apps on the same device.
2. infecting system software with viruses or malware.
3. attempting to take over a device and use it to spread spam.
4. forcing the mobile app to process inordinately large numbers of transactions.
5. storing inordinately large quantities of data on the device.

Creating Weighted Device Platform Matrix

- To mirror real-world conditions, the demographic characteristics of testers should match those of targeted users, as well as those of their devices.
- A ***weighted device platform matrix*** (WDPM) helps ensure that test coverage includes each combination of mobile device and context variables.

1. list the important operating system variants as the matrix column labels.
2. list the targeted devices as the matrix row labels.
3. assign a ranking (for example, 0 to 10) to indicate the relative importance of each operating system and each device.
4. compute the product of each pair of rankings and enter each product as the cell entry in the matrix.

Weighted Device Platform Matrix

TABLE 21.1 Weighted Device Platform Matrix

		OS1	OS2	OS3
	Ranking	3	4	7
Device1	7	N/A	28	49
Device2	3	9	N/A	N/A
Device3	4	12	N/A	N/A
Device4	9	N/A	36	63

Testing AI Systems

- ***Static testing*** is a software verification technique that focuses on review rather than executable testing. It is important to ensure that human experts agree with the ways in which the developers have represented the information and its use in the AI system.
- ***Dynamic testing*** for AI systems is a validation technique that exercises the source code with test cases. The intent is to show that the AI system conforms to the behaviors specified by the human experts.
- ***Model-based testing*** (MBT) a black-box testing technique that uses the requirements model (user stories) as the basis for the generation of test cases from the behavioral model.

Testing Virtual Environments

- ***Acceptance tests*** are a series of specific tests conducted by the customer to uncover product errors before accepting the software from the developer.
- An ***alpha test*** is conducted at the developer's site by a representative group of end users. The software is used in a natural setting with the developer "looking over the shoulder" of the users and recording errors and usage problems.
- A ***beta test*** is conducted at one or more end-user sites. Unlike alpha testing, the developer generally is not present. The customer records all problems that are encountered and reports these at regular intervals.

Usability Test Categories 1

Interactivity. Are interaction mechanisms (for example, pull-down menus, buttons, widgets, inputs) easy to understand and use?

Layout. Are navigation mechanisms, content, and functions placed in a manner that allows the user to find them quickly?

Readability. Is text well written and understandable?9 Are graphic representations easy to understand?

Aesthetics. Do layout, color, typeface, and related characteristics lead to ease of use? Do users "feel comfortable" with the look and feel of the app?

Display characteristics. Does the app make optimal use of screen size and resolution?

Usability Test Categories 2

Time sensitivity. Can important features, functions, and content be used or acquired in a timely manner?

Feedback. Do users receive meaningful feedback to their actions? Is the user's work interruptible and recoverable when a system message is displayed?

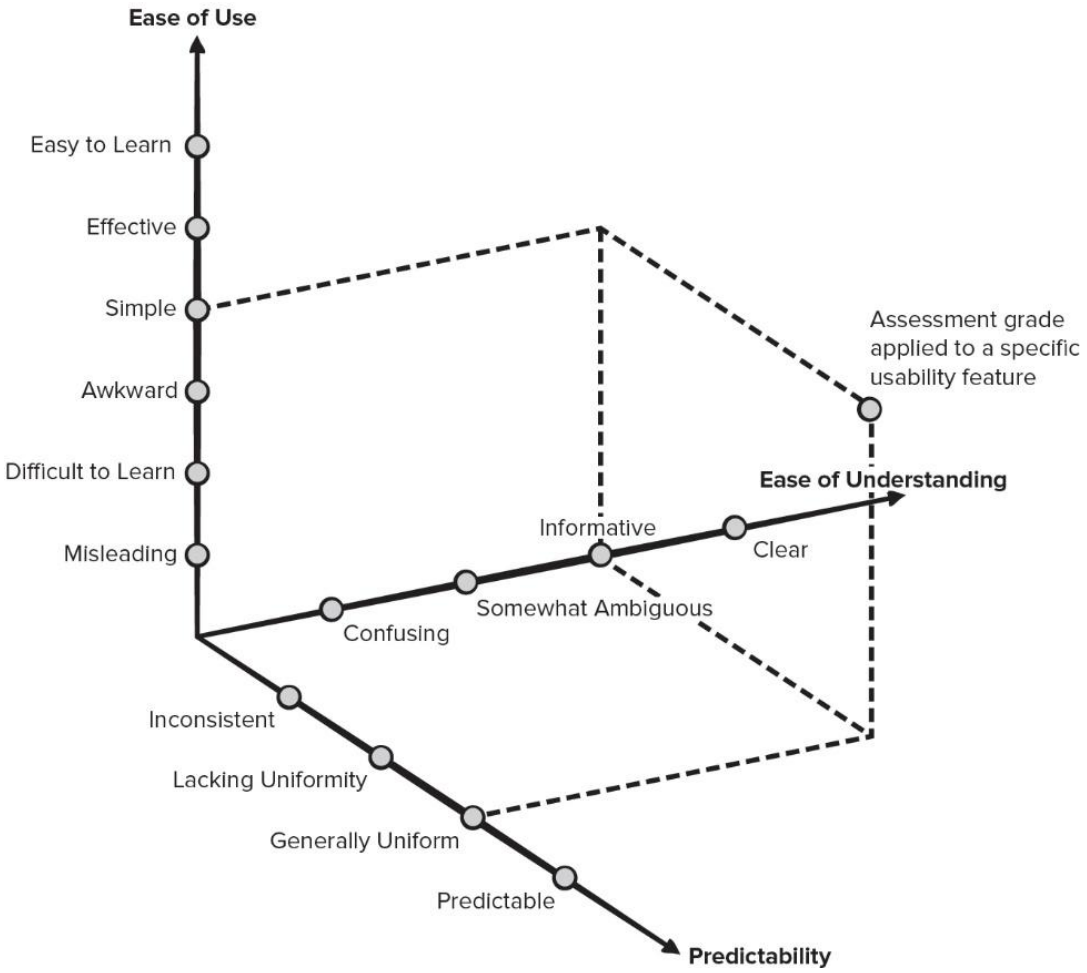
Personalization. Does the app tailor itself to the specific needs of different user categories or individual users?

Help. Is it easy for users to access help and other support options?

Accessibility. Is the app accessible to people who have disabilities?

Trustworthiness. Are users able to control how personal information is shared? Does the app make use of personal information without user permission?

Qualitative Usability Assessment



Access the text alternative for slide images.

Accessibility Testing

- Ensure that non-text screen objects are also represented by a text-based description.
- Verify that color is not used exclusively to convey information to the user.
- Demonstrate that high contrast and magnification options are available for visually challenged users.
- Ensure that speech input alternatives have been implemented to accommodate users that may not be able to manipulate a keyboard, keypad, or mouse.
- Demonstrate that blinking, scrolling, or auto content updating is avoided to accommodate users with reading difficulties.

Playability Testing

- ***Playability*** is the degree to which a game or simulation is fun to play and usable by the user/player.
- Playability testing should be part of the usability testing for the virtual environment created by a MobileApp.
- Expert review should be supplemented by playability tests conducted by representative end users, as you might do for a beta or acceptance test.
- In a typical play test, the user might be given general instructions on using the app and the developers would then step back and observe players use of the game without interruption.
- The developers are looking for places in the where the player does not know what to do next.
- The players may be asked to complete a survey on their experience once they are done with the play test.

Documentation Testing

- Errors in help facilities or online program documentation can devastating to the acceptance of the program.
- Documentation testing should be an important part of every software test plan.

1. The first phase, technical review examines the document for editorial clarity.
2. The second phase, a live test, uses the documentation in conjunction with the actual program.

Because learning changes everything.®

www.mheducation.com